

Resilient SOC Case File 003

From Detection to Automated Response — SOAR Integration Lab

- [Resilient SOC — Case File #003](#)
- [Operation Autopilot: From Detection to Automated Response](#)
 - [SOAR Integration Lab — Technical Build & Validation Guide](#)
 - [Case Metadata](#)
 - [Executive Summary](#)
 - [Objectives](#)
- [1. Target Architecture](#)
 - [1.1 Original specification \(reference\)](#)
 - [1.2 What has actually been exercised in this lab session](#)
- [2. Actual Lab Environment](#)
 - [2.1 Deployed software versions](#)
 - [2.2 Actual service addresses](#)
 - [2.3 Directory layout](#)
- [3. Docker Stack](#)
 - [3.1 Services](#)
 - [3.2 Startup](#)
 - [3.3 Validation](#)
 - [3.4 Known issue — Elasticsearch low-disk watermark](#)
- [4. TheHive](#)
 - [4.1 Installation & access](#)
 - [4.2 Cortex integration](#)
 - [4.3 Case and observable reuse](#)
- [5. Cortex](#)
 - [5.1 API structure](#)
 - [5.2 Authentication](#)
 - [5.3 TestAnalyzer](#)
- [6. Analyzer Repository / IPinfo](#)
 - [6.1 Situation](#)
 - [6.2 Resolution](#)
 - [6.3 Missing dependency](#)
 - [6.4 Outcome](#)
- [7. Shuffle](#)
 - [7.1 Deployment](#)
 - [7.2 Workflow](#)
 - [7.3 Built-in Cortex action failure](#)
 - [7.4 Resolution](#)
- [8. Shuffle → Cortex Custom Action](#)
 - [8.1 Working configuration](#)
 - [8.2 Failures encountered en route to a working call](#)
- [9. TheHive 2 — Dynamic Observable Retrieval](#)
 - [9.1 Purpose](#)
 - [9.2 Validation of the node itself](#)
 - [9.3 Wiring to Cortex](#)
- [10. Current Blocker \(read this before touching the workflow\)](#)
 - [10.1 Required troubleshooting steps \(for whoever picks this up next\)](#)
- [11. Webhook Architecture \(design target, not yet executed\)](#)
 - [11.1 How the real implementation would work](#)
- [12. Splunk Correlation Search \(reference\)](#)
 - [12.1 What it detects](#)
 - [12.2 MITRE ATT&CK mapping](#)
 - [12.3 Status](#)
- [13. Target Playbook \(design\)](#)

- [14. Human-in-the-Loop — Why It's There On Purpose](#)
- [15. Response Action Reference — REFERENCE / LAB ONLY](#)
- [16. Metrics](#)
- [17. Troubleshooting Reference](#)
- [18. Screenshot / Evidence System](#)
- [19. Screenshot Instructions for the Team](#)
- [20. Final Evidence Checklist](#)
- [21. Final Acceptance Criteria](#)
 - [Level 1 — Core SOAR Integration](#)
 - [Level 2 — Full Production-style Pipeline](#)
- [22. Team Handoff — Start Here](#)
- [23. Case File #004 Roadmap \(future work, not started\)](#)
- [24. Security Considerations](#)
- [25. Package Structure](#)
- [26. Credential Handling](#)

Resilient SOC — Case File #003

Operation Autopilot: From Detection to Automated Response

SOAR Integration Lab — Technical Build & Validation Guide

Case ID: RSOC-CF-003 **Prepared for:** Resilient SOC team **Document type:** Internal SOC engineering documentation **Classification:** Internal training use only — not for distribution outside the team

Case Metadata

Field	Value
Case File	#003 — Operation Autopilot
Preceding cases	#001 (SOC Foundation / Investigation), #002 (Operation Nightfall)
Objective	Extend detection/investigation workflow into orchestrated, enriched, human-approved automated response
Environment type	Internal training/lab environment
Production impact	None — no production systems targeted by any action in this lab

Executive Summary

Case File #003 moves the Resilient SOC training track from **Detection → Investigation** (the model used in Case Files #001 and #002) toward **Detection → Orchestration → Enrichment → Human Approval → Controlled Automated Response**. The lab stands up a SOAR stack — TheHive for case management, Cortex for observable enrichment, and Shuffle for orchestration — and wires them together so that an observable raised in a TheHive case can be automatically enriched via Cortex, with the eventual goal of triggering that enrichment from a Splunk detection via webhook, gating any response action behind analyst approval.

As of this document, the **core SOAR chain has been proven component-by-component**: TheHive and Cortex are integrated and healthy, a working analyzer executes successfully against a real observable, and a Shuffle Custom Action can call Cortex directly and receive a job back. The **dynamic hand-off of the observable value from a TheHive case into a Cortex job inside a live Shuffle workflow** is the current

blocker — Cortex is receiving an empty string instead of the IP address. The **Splunk correlation search that would trigger this pipeline, the webhook receiver, the human-approval step, and the controlled response actions** are documented as designed reference material for the next phase of work, not as completed and executed stages.

This document is written to be handed to another Resilient SOC team member with zero prior context on this specific lab session.

Objectives

1. Stand up TheHive + Cortex + Shuffle as an integrated SOAR stack.
2. Validate TheHive ↔ Cortex connectivity and authentication.
3. Get a working Cortex analyzer (IPinfo) enriching a real observable.
4. Build a Shuffle workflow (SOC-Auto-Response) that pulls an observable from a TheHive case and sends it to Cortex for enrichment.
5. Design (not yet execute) the full production-style pipeline: Splunk detection → webhook → Shuffle → TheHive → Cortex → human approval → controlled response → case update → report.
6. Document every failure encountered and its fix, so the path is repeatable.
7. Produce an evidence trail (screenshots + index) sufficient for a training review.

1. Target Architecture

1.1 Original specification (reference)



This is the **design target** for Case File #003. It is documented in full below (Sections 11–15) as the blueprint the team is building toward. Not every arrow in this diagram has been executed end-to-end — see Section 21 for the honest breakdown of what is validated versus planned.

1.2 What has actually been exercised in this lab session

```
TheHive (existing Case ~122884224, observable 8.8.8.8)
  ↓ [validated manually / via terminal]
Shuffle – TheHive 2 node (retrieves the observable)
  ↓ [node itself validated; hand-off to next node NOT yet validated]
Shuffle – Cortex 1 node (Custom Action → Cortex API)
  ↓ [Custom Action mechanism validated with a static IP; dynamic value currently arrives
empty]
Cortex analyzer job (IPinfo_1_3)
  ↓ [validated independently, directly against Cortex, outside the Shuffle dynamic path]
Enrichment result
```

See `architecture/architecture.md` for the annotated diagram version of both flows.

2. Actual Lab Environment

The original task specification referenced an environment of `SOAR-HOST = 192.168.56.12` and earlier software versions. **Those are reference/original-specification values only.** The environment actually deployed and used for this lab is different, and this document describes the actual environment throughout.

2.1 Deployed software versions

Component	Version
TheHive	5.7.5
Cortex	4.1.0
Elasticsearch	8.19.20
Cassandra	4.1.12
Shuffle	Separate deployment (own docker-compose stack)
Docker Engine	28.5.2+dfsg4
Docker Compose	2.40.3-3

2.2 Actual service addresses

```
TheHive:  http://192.168.1.11:9000
Cortex:   http://192.168.1.11:9001/cortex
Shuffle:  http://192.168.1.11:3001
```

Note the Cortex base path includes the `/cortex` context — this matters and is a recurring source of 404s (see Section 17).

2.3 Directory layout

```
~/resilient-soar                # project root
~/resilient-soar/docker/testing # TheHive/Cortex/Elasticsearch/Cassandra docker-
compose stack
~/shuffle                       # Shuffle docker-compose deployment (separate)
```

3. Docker Stack

3.1 Services

The `~/resilient-soar/docker/testing` compose stack runs:

- **Cassandra** — backing datastore for TheHive
- **Elasticsearch** — backing datastore/index for Cortex (and used by TheHive)
- **Cortex** — observable analysis engine
- **TheHive** — case management platform
- **nginx** — reverse proxy in front of the stack
- **Mailhog** — mail capture for notification testing

Shuffle runs as its own compose stack under `~/shuffle`, independent of the above.

3.2 Startup

```
cd ~/resilient-soar/docker/testing
docker compose up -d
```

```
cd ~/shuffle
docker compose up -d
```

3.3 Validation

```
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
```

Expected: all containers `Up` / healthy, no restart loops.

3.4 Known issue — Elasticsearch low-disk watermark

During a stack restart, Elasticsearch logged a **low-disk watermark warning**, which put affected indices into a read-only/degraded state risk. The stack recovered once disk pressure eased and Elasticsearch cleared the watermark condition on its own; no data loss was observed. This is flagged here so a future team member does not mistake the warning for a hard failure requiring destructive remediation.

Do not treat this as a reason to run destructive troubleshooting commands. Specifically avoid, as routine troubleshooting steps:

```
docker compose down -v      # destroys volumes / case + analyzer data
docker system prune         # can remove images/volumes still in use
```

If disk pressure recurs, free disk space or adjust the Elasticsearch watermark thresholds — do not reach for volume deletion first.

Status: COMPLETED / VALIDATED — stack builds, starts, and has been confirmed healthy via `docker ps`.

4. TheHive

4.1 Installation & access

TheHive 5.7.5 is deployed via the compose stack above and reachable at `http://192.168.1.11:9000`.

4.2 Cortex integration

TheHive is configured with a Cortex connection pointing at `http://192.168.1.11:9001/cortex` using a Cortex API key (referenced here as `<CORTEX_API_KEY>` — see Section 26).

Connection validation result:

Status: OK

confirmed from TheHive’s Cortex responder/analyzer configuration screen.

Status: COMPLETED / VALIDATED

4.3 Case and observable reuse

Rather than create duplicate cases/observables for this round of E2E testing, the team **reused an existing case and observable**:

Case ID: ~122884224
Observable: 8.8.8.8

This keeps the evidence trail (Section 18) anchored to a single, consistent case throughout the rest of this document.

Status: COMPLETED (case creation itself was done in an earlier session; reuse for this lab is intentional and documented, not an oversight).

5. Cortex

5.1 API structure

Base URL:

`http://192.168.1.11:9001/cortex`

Key routes used in this lab:

GET /cortex/api/status
GET /cortex/api/analyzer
POST /cortex/api/analyzer/:id/run
GET /cortex/api/job/:id/waitreport
GET /cortex/api/job/:id/artifacts

5.2 Authentication

Authorization: Bearer `<CORTEX_API_KEY>`

The real API key is never written into this documentation or any committed file — see Section 26.

Cortex API authentication was validated successfully against `/cortex/api/status` and against the analyzer run endpoints.

5.3 TestAnalyzer

A baseline `TestAnalyzer` was run to confirm the Cortex job pipeline works end-to-end before layering IPinfo on

top:

- Job submitted → **Job status: Success**
- Returned the expected SOAR test message payload

Status: COMPLETED / VALIDATED

6. Analyzer Repository / IPinfo

6.1 Situation

The lab specification called for an analyzer named:

```
IPinfo_1_3
```

The **official Cortex analyzer catalog did not expose an analyzer under that exact name**. The closest available implementation in the catalog was:

```
IPinfo_Details
```

6.2 Resolution

Rather than silently substitute a differently-named analyzer (which would break downstream references that expect `IPinfo_1_3`), the team **created a local analyzer definition named `IPinfo_1_3`**, based on the `IPinfo_Details` implementation.

```
Analyzer name: IPinfo_1_3
Analyzer ID:   IPinfo_1_3_1_0
Worker ID:    c4538ef31090f3530d1f6e2b333ad580
```

6.3 Missing dependency

On first run, the analyzer failed because the Python `requests` package was not present in the analyzer's runtime environment. This was resolved by installing the missing dependency into that environment.

6.4 Outcome

The analyzer was enabled in Cortex and **executed directly (outside Shuffle) with a successful result** against the `8.8.8.8` observable.

The real IPinfo API key is referenced only as `<IPINFO_API_KEY>` in this documentation and in any configuration examples.

Status: COMPLETED / VALIDATED (as a standalone Cortex analyzer, run directly — not yet proven through the full dynamic Shuffle path, see Section 10).

7. Shuffle

7.1 Deployment

Shuffle runs as a separate docker-compose deployment under `~/shuffle`, reachable at

http://192.168.1.11:3001.

7.2 Workflow

The active workflow is:

SOC-Auto-Response

It is intended to integrate with both TheHive (to pull case/observable data) and Cortex (to enrich it), using a mix of built-in app actions and a Custom Action.

7.3 Built-in Cortex action failure

The original plan was to use Shuffle's built-in Cortex app action to trigger an analyzer run. This **failed**: the built-in action called a function named:

post_run_analyzer

which **did not exist in the deployed version of the Cortex app inside this Shuffle instance**. The error returned was:

Function post_run_analyzer doesn't exist, or the App is out of date.

7.4 Resolution

Rather than trying to patch or replace the bundled Cortex app, the team built a **Custom Action** in Shuffle that calls the Cortex REST API directly (documented in full in Section 8). This sidesteps the outdated built-in app entirely and gives full control over the request.

Status: Built-in Cortex app — **NOT USABLE (version mismatch)**. Custom Action replacement — **COMPLETED / VALIDATED** for a static payload (see Section 8); dynamic payload still **IN PROGRESS** (see Section 10).

8. Shuffle → Cortex Custom Action

8.1 Working configuration

Method: POST

URL: http://192.168.1.11:9001/cortex/api/analyzer/<WORKER_ID>/run

Headers:

Authorization: Bearer <CORTEX_API_KEY>

Content-Type: application/json

Body (static test example):

```
{
  "data": "8.8.8.8",
  "dataType": "ip",
  "tlp": 2,
  "force": 1
}
```

8.2 Failures encountered en route to a working call

1. **First attempt failed** — the URL omitted the `/cortex` context path (i.e. pointed at `http://192.168.1.11:9001/api/analyzer/...`), which returned a 404. Cortex is deployed behind a context path and every API call needs `/cortex` in front of `/api/...`.
2. **Second attempt failed** — the request omitted `Content-Type: application/json`, which Cortex rejected (400).
3. **Final, corrected request returned HTTP 200**, and a Cortex job was created successfully.

Status: COMPLETED / VALIDATED — for a hardcoded/static `8.8.8.8` payload. This proves the Shuffle → Cortex Custom Action mechanism works; it does not by itself prove the *dynamic* case-driven path (Section 10).

9. TheHive 2 — Dynamic Observable Retrieval

To move from a hardcoded IP to a fully case-driven pipeline, a second TheHive node was added to the `SOC-Auto-Response` workflow:

```
Node name: TheHive 2
```

9.1 Purpose

Retrieve the observable(s) attached to:

```
Case ~122884224
```

specifically the existing observable `8.8.8.8`.

9.2 Validation of the node itself

The `TheHive 2` node was tested independently:

- From a terminal call against the TheHive API directly — returned the expected observable data.
- From inside Shuffle, running just this node — also returned data successfully.

Status: VALIDATED at the node level.

9.3 Wiring to Cortex

The workflow was extended:

```
TheHive 2 → Cortex 1
```

The Cortex 1 node's request body was changed from the static example:

```
{
  "data": "8.8.8.8"
}
```

to a Shuffle dynamic expression intended to pull the value out of `TheHive 2`'s output. Two candidate expressions — both suggested by Shuffle's own autocomplete — were tried:

```
$thehive_2.body.0.data
$thehive_2.body.1.data
```

Neither has yet been confirmed as the *correct* mapping — see Section 10.

10. Current Blocker (read this before touching the workflow)

This is the single most important open item in this lab.

During a full run of SOC-Auto-Response, the Cortex 1 node received:

```
{  
  "data": ""  
}
```

instead of the expected:

```
8.8.8.8
```

So: **dynamic observable retrieval is validated at the TheHive 2 node level, but the final variable mapping into Cortex 1 is still under troubleshooting.** It is not correct to describe this as “dynamic extraction completed successfully” — it has not.

10.1 Required troubleshooting steps (for whoever picks this up next)

1. Open the latest SOC-Auto-Response execution in Shuffle.
2. Open the TheHive 2 node’s raw output for that execution.
3. Verify the exact JSON structure Shuffle actually returned (don’t assume it matches the two candidate expressions above).
4. Confirm whether the body field genuinely contains the IP address, and in what shape (object vs array, nested vs flat).
5. Identify the correct array index / key path to the IP value.
6. Update the Cortex 1 node’s dynamic expression to that corrected path.
7. Re-run the workflow.
8. Verify Cortex actually receives 8.8.8.8 in the request body (not an empty string).
9. Verify the resulting Cortex Job status is Success.
10. Verify the IPinfo analysis result content is correct for 8.8.8.8.

Status: IN PROGRESS — not yet resolved as of this document.

11. Webhook Architecture (design target, not yet executed)

The original design calls for event-driven automation:

```
Splunk  
↓  
Webhook  
↓  
Shuffle
```

No webhook has been created or fired in this environment as part of this lab session. This section documents the intended design only.

Expected conceptual endpoint shape (illustrative — no real webhook ID exists yet, and none should be invented):

```
http://<SHUFFLE_HOST>:3001/api/v1/hooks/<webhook_id>
```

11.1 How the real implementation would work

1. A Splunk correlation search fires on a match.
2. Splunk sends an HTTP webhook to the Shuffle endpoint above.
3. Shuffle receives the payload.
4. Shuffle extracts host, user, risk score, timestamp, and observable/IOC from the payload.
5. Shuffle creates or updates a TheHive case with that data.
6. Cortex enrichment executes against the extracted observable.
7. An analyst receives an approval request (Section 14).
8. On approval, the controlled response executes (Section 15).
9. TheHive case is updated with the final disposition.

Status: PLANNED — designed, not executed.

12. Splunk Correlation Search (reference)

```
index=* EventCode=1102
| stats latest(_time) as clear_time by host
| join host [
  search index=* TargetFilename="*.zip"
  | stats latest(_time) as archive_time by host
]
| eval gap = clear_time - archive_time
| where gap < 600
```

12.1 What it detects

Archive activity on a host followed by a **Windows Security Event Log clear (Event ID 1102)** within 10 minutes — a common anti-forensics pattern where an attacker stages/exfiltrates data then clears logs to cover their tracks.

12.2 MITRE ATT&CK mapping

```
T1560.001 – Archive Collected Data: Archive via Utility
T1070.001 – Indicator Removal: Clear Windows Event Logs
```

12.3 Status

The original lab proposal called for this search to run as a real-time or 5-minute scheduled alert with a webhook alert action pointed at Shuffle. **That webhook action has not been configured or fired in this environment.** This SPL is documented as validated *detection logic* (i.e., it is sound SPL for the stated purpose), not as a proven live alert-to-webhook pipeline.

Status: REFERENCE ONLY.

13. Target Playbook (design)

This is the intended shape of the full SOC-Auto-Response playbook once the webhook and approval stages are built. None of the steps below beyond Step 3's enrichment mechanics have been executed live end-to-end.

Step 1 — Receive Webhook

Inputs expected in the payload: - Host - User - Risk Score - Timestamp - Detection Rule - Observable / IP

Step 2 — Create TheHive Case

Reference payload:

```
{
  "title": "Archive-to-LogClear Detected on {{host}}",
  "severity": 3,
  "tags": [
    "T1560.001",
    "T1070.001",
    "resilient-soc"
  ],
  "description": "Correlation rule fired: archive followed by log clear within 10 minutes"
}
```

Step 3 — Cortex Enrichment

Extract the IP/IOC and send it to the IPinfo_1_3 analyzer (this mechanism is proven — see Sections 6 and 8; what's not yet proven is the dynamic hand-off inside a live case-driven run, Section 10).

Step 4 — Human Approval

Example approval prompt:

🚨 Critical Alert: {{host}}
Risk Score: {{risk_score}}
Rule: Archive-to-LogClear
Case: {{thehive_case_url}}

Approve Containment?
YES / NO

Step 5 — Conditional Branch

YES:

Controlled Response
→ Update TheHive = Contained

NO:

Escalate
→ Update TheHive = Escalated

Step 6 — Report Draft

Auto-generate a report containing: Host, User, Timeline, Detection, Observable, Cortex result, Analyst decision, Response taken, Final status.

14. Human-in-the-Loop — Why It's There On Purpose

High-impact response actions — account disable, host isolation, VLAN change, firewall blocking — are **intentionally not** wired to execute automatically off a detection in this design, even in a training environment. The approval gate (Step 4 above) exists because:

- False positives in correlation logic are common; blind auto-response amplifies their blast radius.
- Irreversible or highly disruptive actions (disabling an account, isolating a host) need a human accountable for the decision.
- This is what distinguishes mature SOAR design from “full auto for everything” — the goal of this lab is controlled, auditable automation, not unattended automation.

This is a design principle for the roadmap, not a completed control — since the response stage itself hasn't been built yet (Section 15), there is nothing being auto-executed today either way.

15. Response Action Reference — REFERENCE / LAB ONLY

The following are **reference examples only**, documenting what a containment action *could* look like. They have **not been executed** against any host in this lab, and must **never be run against production assets without explicit authorization**.

```
Disable-ADAccount -Identity "sfahmy"  
Get-ADUser -Identity "sfahmy" | Select-Object Enabled
```

```
Invoke-Command -ComputerName fin-workstation01 -ScriptBlock {  
    New-NetFirewallRule -DisplayName "SOC-Isolation" `  
        -Direction Outbound `  
        -Action Block `  
        -RemoteAddress Any  
}
```

Status: REFERENCE / LAB ONLY — not executed, not for production use without authorization.

16. Metrics

No performance numbers are reported here because none have been measured yet in this environment — inventing percentages would misrepresent the lab's current state. Instead, here is how the team should measure once the pipeline is complete enough to generate real timestamps:

MTTA (Mean Time To Acknowledge):

Detection timestamp → analyst acknowledgement timestamp

MTTC (Mean Time To Containment):

Detection timestamp → containment-after-approval timestamp

Documentation Time:

Detection → Case created → Enrichment complete → Report draft generated

Recommended approach: instrument each stage's timestamp (Splunk alert fire time, TheHive case-created time, Cortex job-complete time, Shuffle approval-response time, response-execution time) once the full webhook path is live, and compute these deltas from real runs rather than estimating them in advance.

Status: Methodology defined; baseline measurement PLANNED (no pipeline runs yet to measure).

17. Troubleshooting Reference

Problem	Symptom	Root Cause	Fix	Validation
Cortex 401	API call rejected, unauthorized	Missing or incorrect Authorization: Bearer header	Set the correct Cortex API key in the header	Retry call, confirm 200 response
Cortex 404	API call not found	URL missing the /cortex context path	Prefix all API calls with /cortex/api/...	Retry call against corrected URL
Cortex 400	Bad request	Missing Content-Type: application/json header	Add the header to the request	Retry call, confirm 200 response
Shuffle Cortex action failure	Function post_run_analyzer doesn't exist, or the App is out of date.	Built-in Shuffle Cortex app references a function not present in the deployed Cortex app version	Replace with a Custom Action calling the Cortex REST API directly	Custom Action returns HTTP 200 and a Cortex job ID
Analyzer failure (IPinfo_1_3)	Analyzer job fails immediately	Missing Python requests dependency in the analyzer runtime	Install requests into the analyzer's environment	Re-run analyzer, confirm Job status = Success
Dynamic IP empty	Cortex 1 receives {"data": ""} instead of 8.8.8.8	Incorrect Shuffle output mapping / expression from TheHive 2 into Cortex 1	Inspect actual TheHive 2 JSON output, correct the expression path (see Section 10)	Re-run workflow, confirm Cortex 1 request body contains 8.8.8.8

18. Screenshot / Evidence System

All evidence lives under screenshots/ using a fixed naming scheme so the Evidence Index (Section 19 / evidence/EVIDENCE-INDEX.md) stays stable as the lab progresses.

```
screenshots/
├─ 01-docker-stack.png
├─ 02-services-healthy.png
├─ 03-thehive-dashboard.png
├─ 04-cortex-dashboard.png
├─ 05-thehive-cortex-ok.png
├─ 06-cortex-analyzers.png
├─ 07-testanalyzer-success.png
├─ 08-thehive-case.png
├─ 09-observable-8.8.8.8.png
├─ 10-ipinfo-analyzer.png
├─ 11-ipinfo-result.png
├─ 12-shuffle-dashboard.png
├─ 13-shuffle-workflow.png
├─ 14-thehive2-config.png
```

- └─ 15-thehive2-output.png
- └─ 16-cortex1-config.png
- └─ 17-cortex-http200.png
- └─ 18-cortex-job-success.png
- └─ 19-analysis-result.png
- └─ 20-final-e2e.png
- └─ 21-webhook-config.png
- └─ 22-splunk-alert.png
- └─ 23-human-approval.png
- └─ 24-thehive-final-status.png
- └─ 25-report-draft.png

Example entry format:

Figure 15 – TheHive 2 Output
File: 15-thehive2-output.png

Purpose:

Proves that Shuffle successfully retrieved the Case observables.

Important:

Hide API keys/tokens/passwords before adding the image.

The full figure-by-figure list with “what it proves” and “where it belongs” lives in `evidence/EVIDENCE-INDEX.md`. **Real screenshots were not supplied for this documentation pass** — the `screenshots/` folder ships with placeholder images and the index marked accordingly. See that file for exact status per image. No evidence is fabricated or claimed as captured when it was not.

19. Screenshot Instructions for the Team

1. Open the relevant UI (TheHive, Cortex, Shuffle, Splunk, terminal, etc.).
2. Capture the screenshot.
3. Crop unnecessary areas (keep it to the relevant panel/result).
4. **Hide secrets** — API keys, tokens, cookies, passwords — before saving.
5. Save the file using the exact required filename from Section 18.
6. Place the image inside `screenshots/`.
7. Update the corresponding row in `evidence/EVIDENCE-INDEX.md` (status → Captured, add notes).
8. Rebuild/re-zip the documentation package if a distributable copy is needed.

20. Final Evidence Checklist

See `evidence/VALIDATION-CHECKLIST.md` for the maintained, checkable version. It covers, at minimum:

Docker, Elasticsearch, Cassandra, TheHive, Cortex, TheHive/Cortex connection, Analyzer, TestAnalyzer, Case, Observable, IPinfo, Shuffle, TheHive 2, Cortex 1, HTTP 200, Cortex Job, Analysis Result, Webhook, Splunk Alert, Human Approval, Response, TheHive Final Status, Report.

21. Final Acceptance Criteria

Level 1 — Core SOAR Integration

```
TheHive Observable
↓
Shuffle
↓
Cortex
↓
Cortex Job
↓
Analysis
```

Status: VALIDATED for the component chain when driven manually / with a static payload. The fully dynamic, case-driven version of this same chain (TheHive 2 → Cortex 1 inside one live workflow run) is **IN PROGRESS** due to the empty-`data` mapping issue in Section 10.

Level 2 — Full Production-style Pipeline

```
Splunk
↓
Webhook
↓
Shuffle
↓
TheHive
↓
Observable
↓
Cortex
↓
Enrichment
↓
Analyst Approval
↓
Controlled Response
↓
TheHive Update
↓
Report
```

Status: PLANNED. Webhook, Splunk alert action, human approval step, and controlled response have not been built or executed in this environment.

22. Team Handoff — Start Here

If you are picking this lab up from here, in order:

1. **Do not delete** the current `SOC-Auto-Response` workflow — it contains the working Custom Action and the in-progress `TheHive 2` node.
2. Verify all services are up (`docker ps` in both compose directories).
3. Open `SOC-Auto-Response` in Shuffle.
4. Inspect the latest execution.
5. Open the `TheHive 2` node.
6. Inspect its exact output structure (don't assume — check).
7. Fix the dynamic mapping into `Cortex 1` per Section 10.
8. Confirm Cortex actually receives `8.8.8.8`.
9. Confirm the Cortex Job status is `Success`.

10. Confirm the IPinfo analysis result is correct.
11. Implement the Splunk webhook trigger (Section 11).
12. Add the human-approval step (Section 14).
13. Test the controlled response path in a safe/lab-only context (Section 15).
14. Capture screenshots per Section 19 and drop them into `screenshots/`.
15. Update `evidence/EVIDENCE-INDEX.md` and `evidence/VALIDATION-CHECKLIST.md`.

23. Case File #004 Roadmap (future work, not started)

Case File #004 — Threat Intelligence Integration

Potential integrations under consideration: - MISP - AbuseIPDB - Additional IOC intelligence sources

Target shape:

```
Detection
↓
Response
↓
Threat Intelligence
```

None of this is deployed. It is listed here purely as forward planning for the next case file.

24. Security Considerations

- No real API keys, passwords, tokens, cookies, or SSH private keys appear anywhere in this document or the accompanying package — see Section 26 for the placeholder convention used throughout.
 - The response actions in Section 15 are reference-only and must never be run against production identities or hosts without explicit, authorized change control.
 - If any real credential was displayed on-screen during hands-on lab work (e.g., visible in a terminal or UI during screenshot capture), **rotate/revoke it** after the lab session, and confirm the corresponding screenshot has that value redacted before it's added to `screenshots/`.
 - The lab environment (`192.168.1.11`) is an internal, non-production network range for this training track.
-

25. Package Structure

```
Resilient-SOC-Case-File-003/
|
├── README.md
├── TEAM-NOTES.md
├── SOAR-Lab-Guide.md
├── SOAR-Lab-Guide.pdf
|
├── architecture/
|   ├── architecture.md
|   └── architecture.png
|
├── screenshots/
|   └── 01-docker-stack.png
```

```
| | | ...
| | | 25-report-draft.png
| |
| | commands/
| | | lab-commands.txt
| |
| | evidence/
| | | EVIDENCE-INDEX.md
| | | VALIDATION-CHECKLIST.md
| |
| | diagrams/
| | | operation-autopilot-flow.png
```

26. Credential Handling

Real secrets are never written into this package. Anywhere a credential is referenced, one of the following placeholders is used instead:

```
<CORTEX_API_KEY>
<THEHIVE_API_KEY>
<SHUFFLE_API_KEY>
<IPINFO_API_KEY>
```

If a real credential was exposed at any point during lab work (terminal history, an unredacted screenshot, etc.), rotate/revoke it after the lab and note the rotation in `TEAM-NOTES.md`.

End of SOAR Lab Guide — see `TEAM-NOTES.md` for open questions and working notes, and `evidence/VALIDATION-CHECKLIST.md` for the current checkable status of every stage.